银龄无诈：面向老年人的 AI 反诈与数字助老平台

技术文档

参赛团队：黄佳欣　江柏宁　张宇恒

指导教师：夏恬恬

提交日期：2026 年 11 月

1 项目概述

1.1 背景

老年人遇到诈骗时，最大的困难往往不是"不知道有诈骗"，而是在具体那一刻判断不了。一条短信、一个电话、一个陌生人发来的链接，年轻人可以截图问朋友，老人往往只能自己拿主意。等家人发现，钱已经转出去了。

我们在社区做过一次小范围访谈，问了六位 65 岁以上的老人三个问题：最近一个月有没有收到过可疑短信、收到之后怎么处理、有没有人教过怎么判断。六个人都收到过，处理方式分别是"删掉"、"打电话问儿子"、"没管它"、"看不懂就没理"。没有人能说清楚自己为什么判断它是诈骗。有一位阿姨说得很直接："我不认识字里那些道道，看着像是真的。"

这说明两件事：判断能力是缺口，而现有的反诈宣传大多是"警惕诈骗"式的口号，缺少针对某一条具体内容的即时判断。

1.2 我们做了什么

银龄无诈是一个面向老年人的 Web 应用，核心是一次针对单条可疑内容的即时判断：老人把短信、聊天记录或者截图放进来，系统告诉它这像不像诈骗、哪里可疑、接下来该做什么。除此之外，还提供了老年人日常最容易卡住的几件事的分步指南（挂号、医保、社保认证、缴费等）和一堂教手机操作的课。

项目由三部分组成：

部分	形态	作用
前端	5 个静态页面，原生 HTML/CSS/JS	大字体界面、本地规则引擎、语音朗读与输入
后端	Python + Flask + SQLite	大模型调用、RAG 检索、语音合成、错误追踪
AI	通过 SiliconFlow 接入 DeepSeek-V3.2、Qwen3-VL、bge-m3、CosyVoice2	语义判断、截图识别、语音合成

线上地址：https://czb.overpass.top

1.3 边界

有几件事我们明确没有做，这里先说清楚：

- 不做医疗建议。反诈结果只做风险提示，不给出医疗或法律结论。
- 不做真实报警联动。页面上的 110、96110 是可拨号的链接，不代拨、不上报。

- 不需要登录。全站没有注册入口，打开就能用，这是刻意的，理由见 8.1。
- 语音识别留在浏览器。后端做不了，原因见 4.6。

2 需求分析

2.1 用户特征带来的硬约束

老年用户和普通用户在交互上的差异，不是"字大一点"就能解决的。我们整理了四条直接影响设计的约束：

视力。老花、白内障、对比度敏感度下降。默认字号必须比常规 Web 应用大，而且不能只放大标题，正文、按钮、输入框要一起放大。我们设了三档字号，默认就是"大字"。

短期记忆。多步操作容易忘掉上一步。所有分步指南都做成"一步一屏、带序号"，并且支持朗读，让用户可以边听边做，不用盯着屏幕记。

操作精度。手指点击容易偏，误触率高。主要按钮最小高度 52px，重点操作 60px，按钮之间留足间距。

信任与情绪。老人被骗之后的第一反应往往是羞耻，不愿意说。文案上我们刻意不用"老年人模式""防骗警示"这类居高临下的词，识别结果页固定写一句"被骗不是您的错"。

2.2 功能需求

模块	需求	优先级
反诈识别	粘贴文本或截图，给出风险等级、可疑点、下一步建议	高
反诈识别	断网或后端不可用时仍能给出结果	高
反诈识别	结果可以朗读、可以转发给家人	中
便民服务	挂号、医保、社保、缴费等分步指南，支持搜索与朗读	高
手机课堂	6 节基础课 + 反诈测验，记录学习进度	中
显示设置	字号、高对比度、减少动画、自动朗读	高
语音	语音输入（说话代替打字）、语音朗读	中

2.3 非功能需求

弱网可用。老人的手机网络和家里的 Wi-Fi 都不稳定。前端的反诈识别必须在没有网络时也能工作，这一条直接决定了我们要保留一套完整的本地规则引擎。

隐私。最初的设计是"所有内容只在本地分析，不上传"。接入大模型之后这个承诺只能部分保留：文本分析必须上传，本地分析降级为兜底。页面文案随之修改，现在写的是"内容仅用于本次判断"。

响应速度。老人对"转圈"的容忍度很低，超过十秒基本会放弃。这决定了我们必须做流式输出，而不是等完整结果（见 4.5）。

可解释。只给一个"高风险"没有用，老人需要知道"哪里不对"。所以每次判断都必须给出 2 到 4 条具体依据。

3 系统架构设计

3.1 总体结构

```
浏览器
  │  HTTPS 443
  ▼
nginx ─┬─ /        ──► 127.0.0.1:4173   node server.js（静态页面）
       └─ /api/    ──► 127.0.0.1:14087  gunicorn（Flask 应用）
                                            │
                                            ├─ SQLite（知识库向量、运行日志）
                                            └─ SiliconFlow API（大模型、向量、语音）
```

选择 nginx 反向代理而不是让前端直接跨端口调后端，理由有三个：同源之后不存在 CORS 和 Cookie 跨域问题；对外只暴露 443 一个端口，后端进程只监听回环地址；静态资源与 API 可以分别配置超时和缓冲策略。

前端没有用构建工具。五个页面都是普通 HTML，脚本用原生 `<script>` 引入，没有模块打包。这样做的好处是部署时不需要构建步骤，改一行 CSS 传上去就生效；代价是没有组件复用，公共结构靠模板复制，这个取舍在 7.2 里承认。

3.2 前端结构

```
index.html       首页：问候、快捷入口、能力介绍
fraud.html       反诈识别
services.html    便民服务（19 项指南）
learn.html       手机课堂（6 节课 + 测验）
profile.html     我的：显示设置、亲情联系人、本机数据

js/app.js        公共模块：设置、图标、朗读、弹窗、本地存储
js/fraud-engine.js  本地反诈规则引擎（19 类，765 行）
js/fraud.js      识别页交互与流式对接
js/home.js / services.js / learn.js / profile.js  各页面逻辑
```

公共能力都挂在 `window.YinLing` 上，各页面通过它取设置、弹提示、调用朗读。这样避免了一个页面一个工具函数的重复。

3.3 后端结构

Flask 应用按职责分层，没有把逻辑堆在路由里：

层	文件	职责
入口	app.py	建表、注册蓝图、CORS、异常映射、兜底路由
配置	config.py	全部配置从环境变量读取，含 `.env` 加载
数据	db.py / store.py	连接管理、建表迁移、所有读写与取值范围校验
响应	responses.py	统一 `status/msg/trace_id` 外壳、业务码到 HTTP 码的映射
接口	api_analyze.py / api_tts.py	只做参数校验、限流与调用
AI	ai.py / rag.py / analyzer.py / knowledge.py	上游客户端、检索、Prompt 与解析

所有接口返回固定三个字段：`status`（业务码）、`msg`（可直接展示的中文提示）、`trace_id`（用于日志排查）。业务码和 HTTP 状态码分离，前端只判断 `status`，HTTP 码留给网关和监控。

失败请求会写入 `ERR_TRACE` 表，记录路径、方法、IP、响应体，便于按 `trace_id` 复盘。

3.4 数据模型

SQLite，服务于两部分：RAG 知识库和运行日志。

表	主要字段	说明
KNOWLEDGE	ID、TITLE、CONTENT、VECTOR	RAG 知识库，27 条，向量以 JSON 存 1024 维
ERR_TRACE	TRACE_ID、PATH、METHOD、IP、STATUS、RET_CONTENT	失败请求追踪，按 trace_id 复盘
CIPHER_SEEN	DIGEST、SEEN_AT	短时间内的重复请求去重

建表语句在 `db.py`，首次启动自动执行，旧库会做一次结构迁移。

3.5 部署拓扑

单台云服务器，Ubuntu 24.04。nginx 终止 TLS，两个应用进程常驻在 `screen` 会话里，分别监听 4173（node 静态服务）和 14087（gunicorn）。证书是泛域名 `*.overpass.top`。

4 AI 技术方案

这一节是重点，也是我们花时间最多的地方。

4.1 整体链路

```
文本 / 截图
   │
   ▼
POST /api/analyze
   │
   ├─► RAG：bge-m3 向量化 → 余弦粗排 Top8 → bge-reranker 精排 Top4
   │        └─ 命中条目拼进 Prompt
   ▼
DeepSeek-V3.2（stream=true, response_format=json_object）
   │
   ├─► 逐 chunk 解析 → 抽出 hint 字段 → SSE 推给前端（逐字显示）
   └─► 生成结束 → 解析完整 JSON → SSE 推 result（渲染卡片）
```

一个请求同时做到"流式体验"和"结构化输出"，没有拆成两次调用。

4.2 模型选型

我们不是凭印象选的，写了一个探测脚本把候选模型在同一个任务上跑了一遍（`max_tokens=250`，同一段诈骗短信）：

模型	耗时	输出 token	结论
DeepSeek-V3.2	10.1 s	148	采用
Qwen3.5-35B-A3B	21.9 – 29.9 s	1354	慢 2 – 3 倍，输出失控
Qwen3.5-9B	22.6 – 34.2 s	1493	同上
Qwen3.5-27B	90 s 超时	—	不可用

Qwen 系列有个现象值得记录：`max_tokens` 设为 250，实际输出 1300 多个 token，参数没有生效，估计是默认开了思考链。对这个任务来说，思考链带来的准确率提升，远抵不上延迟和成本的增加。

其余模型：

用途	模型	实测
截图理解	Qwen3-VL-8B-Instruct	真实聊天截图 OCR 准确，2.8 s
向量化	bge-m3	1024 维，0.3 s
重排	bge-reranker-v2-m3	返回下标与相关性分数，0.3 s
语音合成	CosyVoice2-0.5B	返回 mp3，1.3 s

4.3 Prompt 设计

三条约束是刻意加的，每一条都对应一个具体问题。

第一，只输出 JSON，不开代码块。 配合 `response_format=json_object`，实测返回即合法 JSON。服务端仍然保留一层容错：剥掉意外的 ``` 标记、截取首尾大括号再解析一次。

第二，字段顺序固定为 `hint → level → score → reasons → advice`。 这是全篇最关键的一条。模型按顺序生成 token，把"给老人看的那句话"放在第一个字段，前端才能在这句话还没写完时就开始显示。如果不约束顺序，模型可能先写 `reasons`，流式就失去意义了。

第三，把判断阈值写死。 涉及转账、验证码、屏幕共享、陌生链接的，至少按 medium 处理；出现"安全账户""保密不要告诉家人"的，直接按 high 处理。这样输出稳定、可解释，不是每次靠模型自由发挥。

系统提示里还固定了两句话：不能替代警方判断，不要制造恐慌。实测下来输出语气确实平和很多。

4.4 RAG 设计

知识库共 27 条，来源是项目已有的内容，不是另起炉灶：前端规则引擎里 19 类骗术的判定逻辑与解释，加上页面里已经写过的官方渠道与热线（96110、110、12381、12333、12393、12315）。

每条按"名称 + 典型话术 + 识别要点 + 正确做法"的结构写，不是关键词表。这样向量检索命中的是语义，比如"让我把钱转到安全账户"能命中 `safe-account`，而不是只匹配"转账"两个字。

检索流程：全部条目用 bge-m3 向量化后存进 SQLite，查询时先算余弦相似度取前 8 条，再交给 reranker 精排到前 4 条，拼进 Prompt。

为什么不用向量粗排的结果直接进 Prompt？反诈话术高度相似，很多都提到"转账"，粗排容易把"高收益投资"排在"冒充公检法"前面。重排只多花 0.3 秒，但对准确性影响明显。实测中"冒充公检法"那条命中的是 `authority / secrecy / safe-account`，排序是对的。

降级分三层：embedding 失败退化为字符重合度匹配，reranker 失败退回复用向量粗排结果，整个 RAG 失败就不带参考资料让模型按常识判断。三层都不影响主流程。

4.5 流式设计

接口返回 `text/event-stream`，事件序列固定：

事件	时机	前端用途
start	立即	显示"正在分析"
context	RAG 检索完成，约 0.5 s	可显示"已匹配 3 条相似骗术"
hint	随生成增量，多次	追加到气泡，逐字显示
result	生成结束	用 detail 渲染风险卡片
done	最后一条	结束加载状态
fallback	失败时替代 result	前端改用本地规则引擎

前端的解析方式是手动切帧，没有用 `EventSource`。原因是 `EventSource` 只支持 GET，发不了 JSON 请求体，也带不了自定义请求头。切帧时兼容 `\n\n` 和 `\r\n\r\n` 两种换行，这一点在测试中被真实踩到过（见 6.4）。

4.6 多模态：为什么做截图、不做语音

多模态这块我们做了取舍。

截图能做，而且应该做。 老人收到可疑短信，最容易的操作是截图发给子女，而不是对着手机描述内容。实测把一张真实的微信群聊截图交给 Qwen3-VL，模型准确读出了群名和活动内容，耗时 2.8 秒。接口接受 base64，前端用 canvas 压到 1440px 再传，体积可控。

语音识别做不了。 我们原本计划把录音上传、后端识别，探测时发现 SiliconFlow 的接口明确返回 `Only text and image_url are supported`，`Qwen3-Omni` 也不接受音频。清单里没有专用 ASR 模型。要做得另外接阿里云或讯飞，涉及资质、审核和一天的开发量，最后没有做。

语音输入仍然保留，走浏览器的 Web Speech API，识别出的文字直接进输入框，再交给后端分析。换句话说，语音只是输入方式，不参与识别本身。

4.7 可控性与降级

四层兜底，从外到内：

1. 服务端没有配置 API Key 时，`/api/analyze` 直接返回 1017，前端用本地规则引擎；
2. 上游超时或报错，服务端发 `fallback` 事件，前端同样回退本地规则；
3. 模型返回的不是 JSON，服务端尝试剥离代码块、截取大括号，仍失败则 `fallback`；
4. JSON 字段非法（level 不在枚举里、score 越界），服务端按分数推导等级、把分数裁到 0 到 100 之间。

前端原有的 19 条正则规则引擎完整保留。它不是历史包袱，而是断网和降级时的唯一判断依据，也是"可控性"的证据。

4.8 成本与限流

单次文本分析约 1000 输入、150 输出 token，按量计费成本可以忽略。需要控制的是调用次数：分析接口按 IP 每分钟 6 次，朗读接口每分钟 10 次，超出返回 1016。

RAG 建库只在第一次调用时做一次（27 条一次 embedding 请求），之后走本地 SQLite。朗读接口对"文字 + 音色 + 语速"做进程内缓存，固定的建议语不会重复计费。

5 核心功能实现

5.1 双引擎的反诈识别

反诈页是项目最核心的页面，也是唯一同时存在两套判断逻辑的地方。

本地引擎守在浏览器里，19 类规则、765 行 JS，包括正则匹配、URL 风险分析（短链、IP 直连、可疑后缀、Punycode、URL 里的 @）、加权求和、原文高亮。它的优点是零延迟、零成本、断网可用，缺点是只能识别"话术特征"，理解不了上下文。

AI 引擎在后面，负责语义判断和依据生成。

两者的衔接不是"二选一"，而是主备：正常情况下调 AI，AI 不可用（没有 Key、超时、返回异常、网络断开）时自动切到本地引擎，并且明确告诉用户"AI 暂不可用，已用本地规则判断"。这个提示很重要，不能让用户以为两套结果是一回事。

实测中四种典型内容的判断如下：

输入	AI 判定	分数	本地引擎
快递理赔 + 陌生链接	high	95	high
冒充公检法 + 安全账户	high	95	high
正常社区健康讲座通知	low	5	low
冒充亲戚借钱	high	85	high

第三行是我们最在意的一条：正常内容没有被误报成诈骗。如果模型什么都说高危，这个功能对老人就是纯噪音。

5.2 适老化设计

字号三档（标准、大字、特大），通过根字号联动，全站生效，刷新后记住。高对比度模式只改颜色变量，不动结构。减少动画模式同时影响 CSS 过渡和脚本里的 `scrollIntoView`。

朗读用浏览器 `speechSynthesis`，语速降到 0.88，优先选中文语音。识别结果、办事步骤、课程内容都可以读。后来补了服务端语音合成接口（见 5.4），音色更自然，但前端还没切过去。

语音输入用 `webkitSpeechRecognition`，`interimResults` 打开，说话时逐字上屏，老人能看到"听懂了"。不支持的浏览器（比如 Firefox）按钮会置灰并提示不可用，不会假装能点。

5.3 便民服务与手机课堂

19 项服务指南和 6 节课的内容都是硬编码在前端的 JSON 结构里，没有走接口。这是有意的：这些内容几乎不变，放本地可以让首屏更快、断网也能看。

每项服务包含步骤、注意事项、需要的证件、预计耗时、建议是否要家人陪同。课程有进度记录和一个小测验，成绩存本地。

5.4 语音合成

`POST /api/tts`，输入文字，返回 mp3。参数支持音色和语速，服务端对"文字 + 音色 + 语速"做进程内缓存，同一段文字第二次请求直接命中，不再调用上游。

这是全站唯一一个成功时返回二进制、失败时返回 JSON 的接口。前端读成 blob 再用 `<audio>` 播放，出错时回退浏览器朗读。

6 测试与部署

6.1 测试策略

我们没有写单元测试框架，而是按"能不能跑通"分了三层：

接口级：55 条端到端断言，用脚本对着真实运行的服务跑，覆盖参数校验、边界值、错误码映射、限流触发，以及重复提交时的幂等处理。

AI 链路：30 条断言，起一个假的上游服务，验证 SSE 帧格式、hint 跨 chunk 拼接、JSON 容错、限流、上游 500 时的降级。这套完全离线，不需要 Key 也不联网，改代码时可以随时跑。

真实接口：另写探测脚本打真实 API，确认模型可用性、流式行为、图片输入、向量维度和语音返回格式。这部分要花钱，只在换模型或改协议时跑。

6.2 实测数据

线上环境（nginx + gunicorn + 真实模型），一条 57 字的诈骗短信：

```
start          0 ms
context      952 ms      命中 refund / fake-service / credentials
hint 首字    2.2-3.3 s
result       6.4-10.0 s
```

首字延迟比裸接口慢，原因是 Prompt 里多了 RAG 的参考资料，输入 token 从 200 涨到 1000 左右，预填充时间变长。这个代价我们接受了，因为它换来的是判断依据更贴合具体骗术。

冷启动（第一次调用需要建向量索引）会多 3 秒左右，索引建好后不再出现。

6.3 部署

单机部署，nginx 只开 443，80 跳转。后端两个进程分别跑在 screen 会话里，用启动脚本拉起，日志落到文件。

nginx 配置里有两个关键点：

- `/api/` 必须 `proxy_buffering off`，否则前端会等到最后一次性收到全部内容，流式白做；
- `proxy_read_timeout` 设为 300 秒，默认 60 秒会把长回答掐断。

后端用 gunicorn 单 worker 多线程（`-w 1 --threads 8`），没有用多进程。原因是 SQLite 写入在多进程下容易锁冲突，限流和缓存也会各自为政。单进程多线程对当前量级足够。

6.4 踩过的坑

记录几个真实修过的故障，这些比"系统运行稳定"更有信息量。

SSE 解析只认 `\n\n`。 上线前用 Node 模拟分片时发现一帧都没解析出来，原因是测试文件被 Windows 文本模式写成了 `\r\n`。虽然实际网络传输是 `\n`，但任何中间层都可能改写换行，最后改成正则兼容两种。

`pkill -f gunicorn` 把自己杀了。 重启后端时用 `pkill -f "gunicorn"`，执行命令的 shell 自身命令行里也含这个字符串，结果连同 shell 一起被杀，进程没拉起来，前端断了三分钟。后来改成按 screen 会话名退出，并写进运维文档。

孤儿进程顶班。 有一次"重启"后接口还能用，但 screen 会话里看不到进程，实际是新进程绑不上端口退出了，旧进程（父进程已是 init）还在服务。这种状态最危险：进程挂了没人能进会话拉起。

6.5 上线前的自查

- 五个页面在桌面、平板、手机三种宽度下无横向溢出；
- 字号"特大"模式下 320–1440px 布局正常；
- 反诈样例覆盖四类骗术加一类正常内容；
- 断网时反诈识别仍可用（走本地引擎）；
- 全站无外部资源请求，除 AI 接口外不依赖第三方域名。

7 总结与展望

7.1 已经做到的

一个能用的、面向具体场景的 AI 反诈工具，在线可访问。判断有依据、结果能朗读、弱网能降级。适老化不是换个大字号，而是从字号、按钮尺寸、文案语气、操作步数四个维度一起改。

AI 部分不是简单调一次接口：检索增强让判断有据可依，流式输出把等待从十秒压到两三秒，四层降级保证模型不可用时功能不消失。

7.2 不足

前端没有组件化。 五个页面共用结构靠复制，改一处要改五处。当时选择不引入构建工具是为了部署简单，但代价在后期维护上体现出来了。如果重来，会考虑用轻量框架加一次构建。

没有 Agent 多步编排。 现在是单轮"检索 + 判断"，没有"分析 → 查证 → 给出处置建议"这样的多步规划，也没有接 MCP 工具。这是评分表里权重不低的一块，也是下一步最该补的。

语音识别受限于平台。 只能依赖浏览器，Firefox 用户用不了。要独立于浏览器，需要自建或接入第三方 ASR。

7.3 下一步

优先级从高到低：

1. 把反诈判断改成 Agent 工作流：判断之后调用查证工具（域名检测、号码库），再生成处置步骤；
2. 接入一两个 MCP 服务，让工具调用标准化；
3. 前端重构成组件化结构，顺手把设计系统沉淀下来；
4. 补 ASR，让不识字或不便打字的老人也能用。

8 创新点说明

8.1 应用创新

把"防诈骗"从宣传变成一次具体判断。 现有反诈产品大多在讲道理、发案例，用户看完仍然不知道自己手里这条是真的还是假的。我们的切入点很小：只判断你手上这一条，给出依据和下一步。这个切口让产品有了明确的使用时机，而不是"装了就忘"。

适老化不是放大字号，而是不设门槛。 全站没有注册入口，打开就能用。老人记不住密码，多一步注册就是一道墙；这也让页面上"无需注册"那句话站得住。

截图优先于语音。 老人遇到可疑内容，最自然的动作是截图发给子女，而不是对着手机描述。所以多模态我们做的是图像理解，不是语音识别。

8.2 技术创新

hint 优先的结构化流式。 结构化输出和流式体验天然冲突：JSON 要完整才能解析，流式要逐字才有意义。我们的做法是在 Prompt 里固定字段顺序，把给人看的那句话放在第一个字段，服务端逐 chunk 正则提取、增量推给前端，等生成结束再整体解析。一个请求同时满足两种需求，没有拆成两次调用。

检索重排两段式。 反诈话术相似度高，纯向量检索容易把相近的骗术排错顺序。加上 reranker 之后，"冒充公检法"能准确命中"安全账户""要求保密"这几条，而不是笼统命中"转账"。

四层降级链。 没有 Key、上游失败、返回非 JSON、字段非法，四种失败各有出口，任何一种都不会让页面白屏。前端保留的 19 条规则引擎是最后一道防线，也是断网场景下唯一的判断依据。

附录 A 接口清单

方法	路径	说明
POST	/api/analyze	AI 反诈分析，SSE 流式返回
POST	/api/tts	语音合成，返回 mp3

两个接口都不需要登录，按 IP 限流；服务端未配置模型 Key 时返回 1017，前端自动降级到本地规则引擎和浏览器朗读。

附录 B 技术栈

层	选型
前端	原生 HTML / CSS / JavaScript，无框架无构建
后端	Python 3.12 + Flask 3.1 + SQLite
应用服务器	gunicorn（1 worker / 8 threads）
反向代理	nginx 1.24
AI 平台	SiliconFlow
模型	DeepSeek-V3.2、Qwen3-VL-8B、bge-m3、bge-reranker-v2-m3、CosyVoice2-0.5B
部署	Ubuntu 24.04，screen 常驻